Department of Information and Communication Technology

Faculty of Technology

University of Ruhuna

Database Management Systems Practicum

ICT 1222

Assignment 02 – Mini Project

Group Number 16

Submitted to: Mr.P.H.P. Nuwan Laksiri

Submitted by: TG/2023/1721 - P.H.T.Sharindi

TG/2023/1781 - H.G.D.I.Shadeep

TG/2023/1748 - R.P.D.Radeeshan

Contents

I. Brief Introduction to the Problem/group project

A faculty of technology needs a system to manage all student information. The main problems are tracking student details, student marks, student attendance and result management.
This task is difficult to do manually. We need to store various scores (like quizzes and final exam scores) and check who is eligible for the final exam. All the results must follow the rules prescribed by UGC Circular No. 12-2024. We need a good system to avoid errors and keep the data safe.

In a manual system, teachers spend too much time adding marks and checking attendance percentages. Also, students cannot easily check their own attendance or final marks quickly.
Also, it is very difficult to create important reports like SGPA and CGPA for everyone.
So as summary, we need to create a solution for a problem like this.

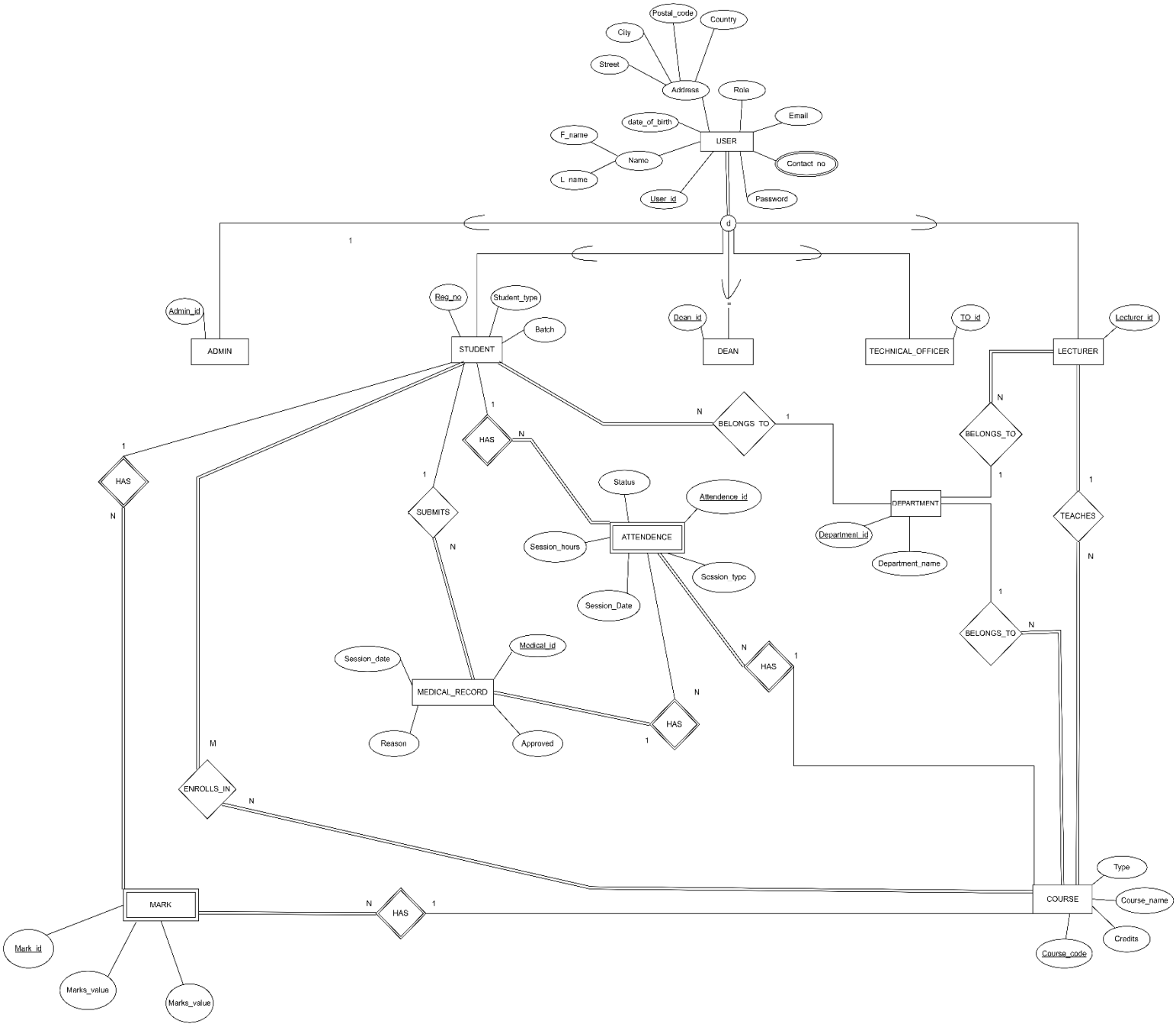
II. Brief Introduction to the Solution

The solution is for this problem is develop a Database Management System. By This system it will automatically manage and calculate all data of the faculty.

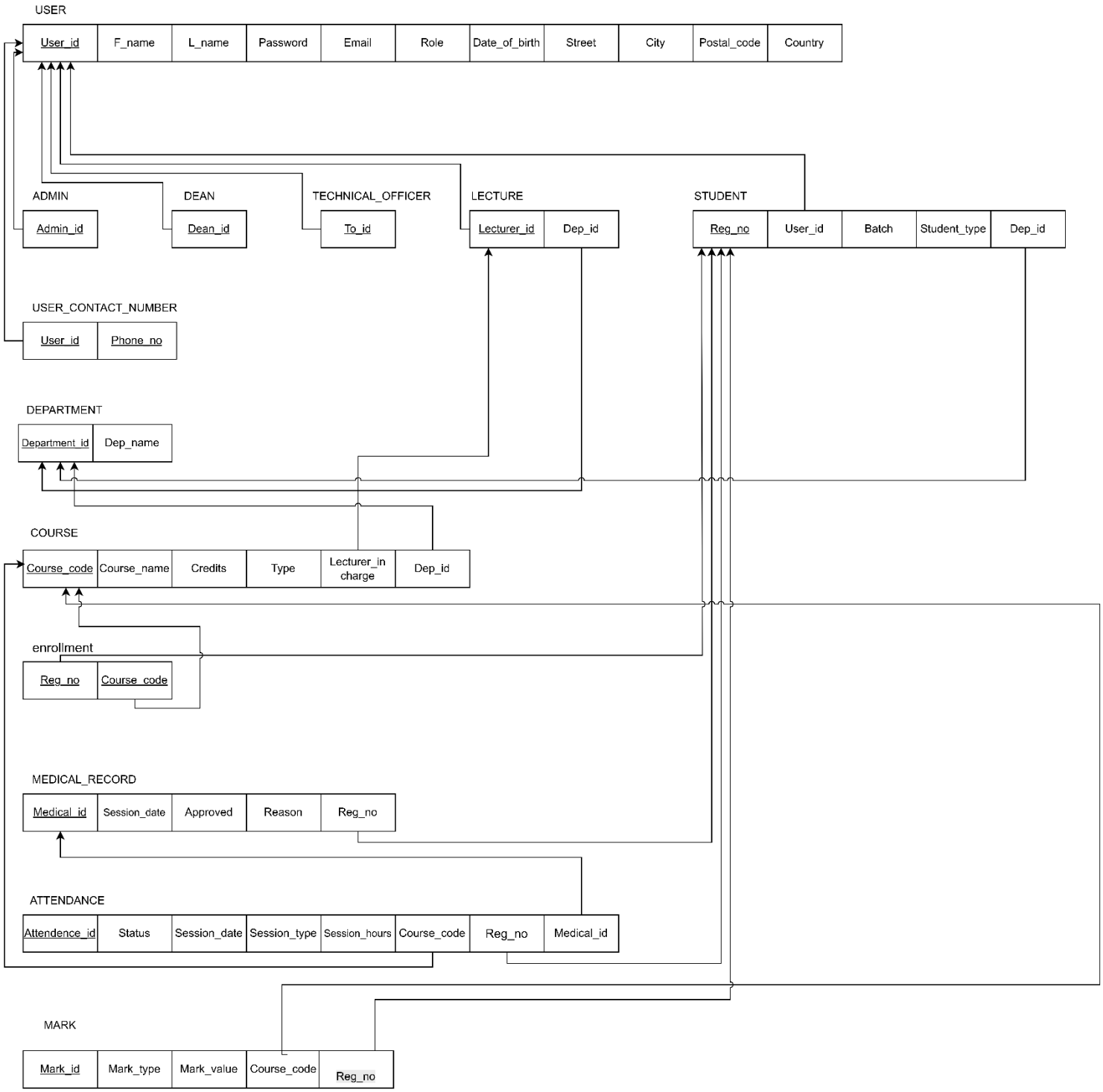
By this solution, it,

- Store all data: Keep all student details, user details, marks, and attendance records, etc.
- Track attendance: Record students' attendance for both theory and practical sessions. It can also record when a student submits a medical note.
- Manage users by setting up separate accounts for various roles, including Admin, Dean, Lecturer, Technical Officer, and Student. Each account type will have only the required access.
- Calculate results: Automatically calculate eligibility for the final exam. Then, it will calculate the student's final grade, SGPA, and CGPA using UGC rules.
- This system helps manage student academic data easily and securely for the entire faculty.

III. Proposed ER/EER diagram



IV. Proposed Relational mapping diagram



V. Table structure of solution

User table –

```
mysql> DESC user;
+--------------+-------------+------+-----+---------+----------------+
| Field        | Type        | Null | Key | Default | Extra          |
+--------------+-------------+------+-----+---------+----------------+
| User_id      | int         | NO   | PRI | NULL    | auto_increment |
| F_name       | varchar(50) | NO   |     | NULL    |                |
| L_name       | varchar(50) | NO   |     | NULL    |                |
| date_of_birth | date       | YES  |     | NULL    |                |
| Street       | varchar(100) | YES |     | NULL    |                |
| City         | varchar(100) | YES |     | NULL    |                |
| Postal_code  | char(50)    | YES  |     | NULL    |                |
| Email        | varchar(100) | YES | UNI | NULL    |                |
| Role         | varchar(50) | YES  |     | NULL    |                |
| Password     | varchar(100) | YES |     | NULL    |                |
+--------------+-------------+------+-----+---------+----------------+
```

Dean table -

```
mysql> DESC dean;
+---------+------+------+-----+---------+-------+
| Field   | Type | Null | Key | Default | Extra |
+---------+------+------+-----+---------+-------+
| Dean_id | int  | NO   | PRI | NULL    |       |
+---------+------+------+-----+---------+-------+
```

Admin table -

```
mysql> DESC admin;
+----------+------+------+-----+---------+-------+
| Field    | Type | Null | Key | Default | Extra |
+----------+------+------+-----+---------+-------+
| Admin_id | int  | NO   | PRI | NULL    |       |
+----------+------+------+-----+---------+-------+
```

Technical officer table-

```
mysql> DESC technical_officer;
+-------+------+------+-----+---------+-------+
| Field | Type | Null | Key | Default | Extra |
+-------+------+------+-----+---------+-------+
| To_id | int  | NO   | PRI | NULL    |       |
+-------+------+------+-----+---------+-------+
```

Student table-

```
mysql> DESC student;
+--------------+-------------+------+-----+---------+-------+
| Field        | Type        | Null | Key | Default | Extra |
+--------------+-------------+------+-----+---------+-------+
| Reg_no       | char(20)    | NO   | PRI | NULL    |       |
| User_id      | int         | YES  | MUL | NULL    |       |
| Batch        | varchar(20) | YES  |     | NULL    |       |
| Student_type | varchar(50) | YES  |     | NULL    |       |
| Dep_id       | char(5)     | YES  | MUL | NULL    |       |
+--------------+-------------+------+-----+---------+-------+
```

Lecture table -

```
mysql> DESC lecturer;
+-------------+---------+------+-----+---------+-------+
| Field       | Type    | Null | Key | Default | Extra |
+-------------+---------+------+-----+---------+-------+
| Lecturer_id | int     | NO   | PRI | NULL    |       |
| Dep_id      | char(5) | YES  | MUL | NULL    |       |
+-------------+---------+------+-----+---------+-------+
```

Attendance table-

```
mysql> DESC attendance;
+---------------+-------------+------+-----+---------+----------------+
| Field         | Type        | Null | Key | Default | Extra          |
+---------------+-------------+------+-----+---------+----------------+
| Attendance_id | int         | NO   | PRI | NULL    | auto_increment |
| Reg_no        | char(20)    | YES  | MUL | NULL    |                |
| Course_code   | varchar(20) | YES  | MUL | NULL    |                |
| Session_date  | date        | YES  |     | NULL    |                |
| Session_type  | varchar(10) | YES  |     | NULL    |                |
| Session_hours | decimal(2,1)| NO   |     | NULL    |                |
| Status        | varchar(10) | YES  |     | NULL    |                |
| Medical_id    | int         | YES  | MUL | NULL    |                |
+---------------+-------------+------+-----+---------+----------------+
```

Course table-

```
mysql> DESC course;
+-------------------+-------------+------+-----+---------+-------+
| Field             | Type        | Null | Key | Default | Extra |
+-------------------+-------------+------+-----+---------+-------+
| Course_code       | varchar(20) | NO   | PRI | NULL    |       |
| Course_name       | varchar(100)| YES  |     | NULL    |       |
| Type              | varchar(50) | YES  |     | NULL    |       |
| Credits           | int         | YES  |     | NULL    |       |
| Lecturer_in_charge| int         | YES  | MUL | NULL    |       |
| Dep_id            | char(5)     | YES  | MUL | NULL    |       |
+-------------------+-------------+------+-----+---------+-------+
```

Deparment table-

```
mysql> DESC department;
+---------------+-------------+------+-----+---------+-------+
| Field         | Type        | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
| Department_id | char(5)     | NO   | PRI | NULL    |       |
| Dep_name      | varchar(50) | YES  |     | NULL    |       |
+---------------+-------------+------+-----+---------+-------+
```

Enrollment table -

```
mysql> DESC enrollment;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| Reg_no      | char(20)    | NO   | PRI | NULL    |       |
| Course_code | varchar(20) | NO   | PRI | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
```

Mark table -

```
mysql> DESC mark;
+-------------+-------------+------+-----+---------+----------------+
| Field       | Type        | Null | Key | Default | Extra          |
+-------------+-------------+------+-----+---------+----------------+
| Mark_id     | int         | NO   | PRI | NULL    | auto_increment |
| Reg_no      | char(20)    | YES  | MUL | NULL    |                |
| Course_code | varchar(20) | YES  | MUL | NULL    |                |
| Marks_type  | varchar(50) | YES  |     | NULL    |                |
| Marks_value | decimal(5,2)| YES  |     | NULL    |                |
+-------------+-------------+------+-----+---------+----------------+
```

Medical_record table -

```
mysql> DESC medical_record;
+--------------+--------------+------+-----+-----------+----------------+
| Field        | Type         | Null | Key | Default   | Extra          |
+--------------+--------------+------+-----+-----------+----------------+
| Medical_id   | int          | NO   | PRI | NULL      | auto_increment |
| Reg_no       | char(20)     | NO   | MUL | NULL      |                |
| Session_date | date         | NO   |     | NULL      |                |
| Reason       | varchar(255) | YES  |     | NULL      |                |
| Session_type | varchar(50)  | YES  |     | NormalDay |                |
| Approved     | tinyint(1)   | YES  |     | 0         |                |
+--------------+--------------+------+-----+-----------+----------------+
```

User contact number table -

```
mysql> DESC user_contact_number;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| User_id  | int         | YES  | MUL | NULL    |       |
| Phone_no | varchar(50) | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
```

VI. Architecture of our solution

The architecture of our solution is a tier architecture. This structure is mainly based on three tires.

They are,

1. Presentation Tier (The Screen) –
    - This is the part of the solution that user sees. This tier shows data and takes our requests.

2. Application Tier (The Brain) –
    - This is the middle layer of the solution that all rules and logic runs.all conditions are run in this layer.

3. Data Tier (The Storage) -
    - This is the database that contains all information. This layer securely stores all data.

With this architecture, we can create a better solution.

VII. Used Tools and technologies

- Draw.io - Used to create the EER diagram and Relational schema.
- MYSQL server, VSCode – for writing structure, query, and other things.
- MS Word – for creating a report.
- GitHub – for version control and collaborating with group members.

VIII. Security measures that you have taken to protect DB

- Access control –
    - Create different user accounts with special permissions
      Like,
      Students can only look at (read) their final marks. They cannot change them.
      Technical Officers can only read, change, or update attendance records.
      Lecturers can work with most data but cannot create new user accounts.
      Only the Admin user has all the power to do everything.

- Use views –
    - By views can show exactly the data that the user needs without displaying all information.

IX. Brief description about DB Accounts/Users and the reasons for creating

- Lecturer
  - Can manage course content and student marks.
  - Can enter and update assessment marks.
  - Can view student attendance for their courses.

- Technical Officer
  - Specifically manages attendance records.
  - Can read, add, and update attendance data.
  - Cannot access marks or personal student information.

- Student
  - Can only view their own final attendance and grades.
  - Cannot modify any data in the system.
  - Can check their academic progress and eligibility status.

- Admin
  - Has full control over the entire database system.
  - Responsible for creating and managing all user accounts.
  - Can modify any table, view, or database structure.
- Dean
  - Can view and modify all academic data.
  - Cannot create or manage user accounts.
  - Can generate reports and review student progress.

Reason is to create that various types of user accounts with different permission is for better control and better security of database. otherwise, it is can control everyone for their own needs.

X. Code snippets to support our work

❖ List of All views

```
mysql> SELECT TABLE_NAME AS view_name
    -> FROM information_schema.VIEWS
    -> WHERE TABLE_SCHEMA = 'faculty_of_technology';
+-------------------------------------------------------------------+
| view_name                                                         |
+-------------------------------------------------------------------+
| vw_all_sgpa_and_cgpa                                              |
| vw_all_stu_all_course_grade_point                                 |
| vw_all_stu_cgpa                                                   |
| vw_all_stu_sgpa                                                   |
| vw_approvedmedical                                                |
| vw_attend_percen_with_medical_with_elig_without_s_type_with_n     |
| vw_attend_percentage_with_medical_with_eligibility_with_name      |
| vw_attend_with_medical_approval                                   |
| vw_ca_status                                                      |
| vw_eligibi_without_medical_with_name                              |
| vw_eligibi_without_medical_with_name_without_s_type               |
| vw_end_exam_elig_basedon_ca_and_atten                             |
| vw_end_status                                                     |
| vw_eng1222_marks                                                  |
| vw_ict1212_marks                                                  |
| vw_ict1222_marks                                                  |
| vw_ict1233_marks                                                  |
| vw_ict1242_marks                                                  |
| vw_ict1253_marks                                                  |
| vw_medicalapplyattendance                                         |
| vw_result_sheet                                                   |
| vw_tcs1212_marks                                                  |
| vw_tms1233_marks                                                  |
| vw_total_ca_end_status                                            |
+-------------------------------------------------------------------+
24 rows in set (0.00 sec)
```

- **views**

```
DROP VIEW IF EXISTS vw_medicalApplyAttendance;

CREATE VIEW vw_medicalApplyAttendance AS
SELECT attendance_id,reg_no,course_code,session_date,session_type,session_hours,status
FROM attendance
WHERE status='Medical';
```

```sql
DROP VIEW IF EXISTS vw_approvedMedical;

CREATE VIEW vw_approvedMedical AS
SELECT Medical_id,Reg_no,session_date,Reason,approved
FROM medical_record
WHERE Approved=TRUE;
```

```sql
DROP VIEW IF EXISTS vw_attend_with_medical_approval;

CREATE VIEW vw_attend_with_medical_approval AS
SELECT
    a.attendance_id,
    a.reg_no,
    a.course_code,
    a.session_date,
    a.session_type,
    a.session_hours,
    CASE
        WHEN a.status = 'Medical' AND m.Approved=TRUE THEN 'Present/M'
        WHEN a.status = 'Medical' AND m.Approved=FALSE THEN 'Absent/M'
        ELSE a.status
    END AS status
FROM attendance a
LEFT JOIN Medical_record m ON
    a.Medical_id = m.Medical_id;
```

```
CREATE VIEW vw_eligibi_without_medical_with_name AS
SELECT
    u.L_name,
    a.reg_no,
    a.course_code,
    a.session_type,
    SUM(CASE WHEN a.status = 'Present' THEN a.session_hours ELSE 0 END) AS attended_hours,
    SUM(a.session_hours) AS total_hours,
    SUM(CASE WHEN a.status = 'Present' THEN 1 ELSE 0 END) AS attended_sessions,
    COUNT(*) AS total_sessions,
    ROUND(
        (SUM(CASE WHEN a.status = 'Present' THEN a.session_hours ELSE 0 END) /
        SUM(a.session_hours)
    ) * 100, 2
    ) AS attendance_percentage,
    CASE
    WHEN
        ROUND(
            (SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) /
            SUM(session_hours)
            ) * 100, 2
    ) >=80 THEN 'Eligible'
       ELSE 'Not Eligible'
    END AS eligibility
FROM attendance a
LEFT JOIN student s ON
s.reg_no=a.reg_no
LEFT JOIN user u ON
u.user_id=s.user_id
GROUP BY a.reg_no, a.course_code, a.session_type
ORDER BY a.reg_no, a.course_code, a.session_type;
```

```
CREATE VIEW vw_attend_percentage_with_medical_with_eligibility_with_name AS
SELECT
 u.L_name,
    a.reg_no,
    a.course_code,
    a.session_type,
    SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END) AS attended_hours,
    SUM(session_hours) AS total_hours,
    SUM(CASE WHEN a.status = 'Present' THEN 1  WHEN a.status='Medical' AND m.approved=TRUE THEN 1 ELSE 0 END) AS attended_sessions,
    COUNT(*) AS total_sessions,
    ROUND( (
        SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.session_hours)
    ) * 100, 2 ) AS attendance_percentage,
    CASE
        WHEN
            SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.session_hours)
                >= 0.8 AND SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) / SUM(session_hours) < 0.8 THEN 'Eligible with medical'
        WHEN
            ROUND( (
                SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.session_hours)) * 100, 2
                ) >= 80 THEN 'Eligible'
        ELSE 'Not Eligible'
    END AS eligibility
FROM attendance a
LEFT JOIN student s ON
s.reg_no=a.reg_no
LEFT JOIN user u ON
u.user_id=s.user_id
LEFT JOIN medical_record m ON
a.medical_id=m.medical_id
GROUP BY a.reg_no, a.course_code, a.session_type
ORDER BY a.reg_no, a.course_code, a.session_type;
```

```sql
CREATE VIEW vw_eligibi_without_medical_with_name_without_S_type AS
SELECT
    u.L_name,
    a.reg_no,
    a.course_code,
    SUM(CASE WHEN a.status = 'Present' THEN a.session_hours ELSE 0 END) AS attended_hours,
    SUM(a.session_hours) AS total_hours,
    SUM(CASE WHEN a.status = 'Present' THEN 1 ELSE 0 END) AS attended_sessions,
    COUNT(*) AS total_sessions,
    ROUND(
        (SUM(CASE WHEN a.status = 'Present' THEN a.session_hours ELSE 0 END) /
        SUM(a.session_hours)
    ) * 100, 2
    ) AS attendance_percentage,
    CASE
        WHEN
            ROUND(
                (SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) /
                SUM(session_hours)
                ) * 100, 2) >=80
        THEN 'Eligible'
        ELSE 'Not Eligible'
    END AS eligibility
FROM attendance a
LEFT JOIN student s ON
s.reg_no=a.reg_no
LEFT JOIN user u ON
u.user_id=s.user_id
GROUP BY a.reg_no, a.course_code
ORDER BY a.reg_no, a.course_code;
```

```sql
CREATE VIEW vw_attend_percen_with_medical_with_elig_without_S_type_with_n AS
SELECT
    u.L_name,
    a.reg_no,
    a.course_code,
    SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END) AS attended_hours,
    SUM(session_hours) AS total_hours,
    SUM(CASE WHEN a.status = 'Present' THEN 1  WHEN a.status='Medical' AND m.approved=TRUE THEN 1 ELSE 0 END) AS attended_sessions,
    COUNT(*) AS total_sessions,
    ROUND( (
        SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.session_hours)
        ) * 100, 2
    ) AS attendance_percentage,
    CASE
        WHEN
            SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.session_hours)
            >= 0.8 AND SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) / SUM(session_hours) < 0.8 THEN 'Eligible with medical'
        WHEN
            ROUND( (SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.session_hours)
            ) * 100, 2) >= 80 THEN 'Eligible'
        ELSE 'Not Eligible'
    END AS eligibility
FROM attendance a
LEFT JOIN student s ON
s.reg_no=a.reg_no
LEFT JOIN user u ON
u.user_id=s.user_id
LEFT JOIN medical_record m ON
a.medical_id=m.medical_id
GROUP BY a.reg_no, a.course_code
ORDER BY a.reg_no, a.course_code;
```

```
CREATE VIEW vw_TMS1233_marks AS
SELECT DISTINCT
    m.Reg_no,
    m.Course_code,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
        WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = m.Reg_no
               AND Session_type = 'ExamTMS1233'
               AND Approved = TRUE) THEN 'MC'
        ELSE IFNULL((SELECT Marks_value FROM mark
                     WHERE Course_code = 'TMS1233'
                       AND Reg_no = m.Reg_no
                       AND Marks_type = 'Quiz_1'), 0)
    END AS `Quiz_1(100%)`,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
        WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = m.Reg_no
               AND Session_type = 'ExamTMS1233'
               AND Approved = TRUE) THEN 'MC'
        ELSE IFNULL((SELECT Marks_value FROM mark
                     WHERE Course_code = 'TMS1233'
                       AND Reg_no = m.Reg_no
                       AND Marks_type = 'Quiz_2'), 0)
    END AS `Quiz_2(100%)`,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
       WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = m.Reg_no
               AND Session_type = 'ExamTMS1233'
               AND Approved = TRUE) THEN 'MC'
        ELSE IFNULL((SELECT Marks_value FROM mark
                     WHERE Course_code = 'TMS1233'
                       AND Reg_no = m.Reg_no
                       AND Marks_type = 'Mid_theory'), 0)
    END AS `Mid_theory(100%)`,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
```

```
CREATE VIEW vw_TCS1212_marks AS
SELECT DISTINCT
    m.Reg_no,
    m.Course_code,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
        WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = m.Reg_no
               AND Session_type = 'ExamTCS1212'
               AND Approved = TRUE) THEN 'MC'
        ELSE IFNULL((SELECT Marks_value FROM mark
                     WHERE Course_code = 'TCS1212'
                       AND Reg_no = m.Reg_no
                       AND Marks_type = 'Quiz_1'), 0)
    END AS `Quiz_1(100%)`,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
        WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = m.Reg_no
               AND Session_type = 'ExamTCS1212'
               AND Approved = TRUE) THEN 'MC'
        ELSE IFNULL((SELECT Marks_value FROM mark
                     WHERE Course_code = 'TCS1212'
                       AND Reg_no = m.Reg_no
                       AND Marks_type = 'Assignment_1'), 0)
    END AS `Assignment_1(100%)`,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
        WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = m.Reg_no
               AND Session_type = 'ExamTCS1212'
               AND Approved = TRUE) THEN 'MC'
        ELSE IFNULL((SELECT Marks_value FROM mark
                     WHERE Course_code = 'TCS1212'
                       AND Reg_no = m.Reg_no
                       AND Marks_type = 'Mid_theory'), 0)
```

```
CREATE VIEW vw_ICT1253_marks AS
SELECT DISTINCT
    m.Reg_no,
    m.Course_code,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
         WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = m.Reg_no
               AND Session_type = 'ExamICT1253'
               AND Approved = TRUE) THEN 'MC'
        ELSE GREATEST(
            IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_1'), 0),
            IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_2'), 0),
            IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_3'), 0)
        )
    END AS `Max_Quiz(100%)`,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
        WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = m.Reg_no
               AND Session_type = 'ExamICT1253'
               AND Approved = TRUE) THEN 'MC'
        ELSE (
            (
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_1'), 0) +
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_2'), 0) +
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_3'), 0)
            )
            -
            GREATEST(
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_1'), 0),
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_2'), 0),
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_3'), 0)
            )
            -
            LEAST(
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_1'), 0),
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_2'), 0),
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1253' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_3'), 0)
            )
        )
    END AS `Second_Max_Quiz(100%)`
```

```
CREATE VIEW vw_ICT1222_marks AS
SELECT DISTINCT
    m.Reg_no,
    m.Course_code,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
         WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = m.Reg_no
               AND Session_type = 'ExamICT1222'
               AND Approved = TRUE) THEN 'MC'
        ELSE GREATEST(
            IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1222' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_1'), 0),
            IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1222' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_2'), 0),
            IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1222' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_3'), 0)
        )
    END AS `Max_Quiz(100%)`,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = m.Reg_no) = 'Suspend' THEN 'WH'
        WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = m.Reg_no
               AND Session_type = 'ExamICT1222'
               AND Approved = TRUE) THEN 'MC'
        ELSE (
            (
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1222' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_1'), 0) +
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1222' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_2'), 0) +
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1222' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_3'), 0)
            )
            -
            GREATEST(
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1222' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_1'), 0),
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1222' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_2'), 0),
                IFNULL((SELECT Marks_value FROM mark WHERE Course_code = 'ICT1222' AND Reg_no = m.Reg_no AND Marks_type = 'Quiz_3'), 0)
```

```
DROP VIEW IF EXISTS vw_total_CA_End_status;
CREATE VIEW vw_total_CA_End_status AS
SELECT
    e.Reg_no,
    e.Course_code,
    ca.CA_Status,
    e.END_Status
FROM
    vw_end_Status e
LEFT JOIN
    vw_ca_status ca
ON
    e.Reg_no = ca.Reg_no
    AND e.Course_code = ca.Course_code;
```

```
CREATE VIEW vw_ENG1222_marks AS
SELECT
    e.Reg_no,
    e.Course_code,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = e.Reg_no) = 'Suspend' THEN 'WH'
        WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = e.Reg_no
               AND Session_type = 'ExamENG1222'
               AND Approved = TRUE) THEN 'MC'
        ELSE IFNULL((SELECT Marks_value FROM mark
                     WHERE Course_code = 'ENG1222'
                       AND Reg_no = e.Reg_no
                       AND Marks_type = 'Assignment_1'), 0)
    END AS `Assignment_1(100%)`,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = e.Reg_no) = 'Suspend' THEN 'WH'
        WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = e.Reg_no
               AND Session_type = 'ExamENG1222'
               AND Approved = TRUE) THEN 'MC'
        ELSE IFNULL((SELECT Marks_value FROM mark
                     WHERE Course_code = 'ENG1222'
                       AND Reg_no = e.Reg_no
                       AND Marks_type = 'Assignment_2'), 0)
    END AS `Assignment_2(100%)`,
    CASE
        WHEN (SELECT Student_type FROM student WHERE Reg_no = e.Reg_no) = 'Suspend' THEN 'WH'
        WHEN EXISTS (SELECT 1 FROM medical_record
             WHERE Reg_no = e.Reg_no
               AND Session_type = 'ExamENG1222'
               AND Approved = TRUE) THEN 'MC'
        ELSE IFNULL((SELECT Marks_value FROM mark
                     WHERE Course_code = 'ENG1222'
                       AND Reg_no = e.Reg_no
```

```
DROP VIEW IF EXISTS vw_CA_Status;
CREATE VIEW vw_CA_Status AS
SELECT
    Reg_no,
    Course_code,
    CA_marks,
    CASE
    WHEN CA_marks = 'MC' THEN 'MC'
    WHEN CA_marks = 'WH' THEN 'WH'
        WHEN Course_code IN ('ICT1222', 'ICT1233', 'ICT1253', 'TMS1233') THEN
            CASE WHEN CA_marks >= 16 THEN 'CA_Pass' ELSE 'CA_Fail' END
        WHEN Course_code IN ('ENG1222', 'ICT1212', 'ICT1242', 'TCS1212') THEN
            CASE WHEN CA_marks >= 12 THEN 'CA_Pass' ELSE 'CA_Fail' END
        ELSE 'Unknown'
    END AS CA_Status
    FROM (
    SELECT
        Reg_no,
        Course_code,
        `Total_CA_marks(30%)` AS CA_marks
    FROM vw_ENG1222_marks

    UNION ALL

    SELECT
        Reg_no,
        Course_code,
        `Total_CA_marks(30%)` AS CA_marks
    FROM vw_ICT1212_marks

    UNION ALL

    SELECT
        Reg_no
```

```
CREATE VIEW vw_end_exam_elig_basedON_CA_and_Atten AS
SELECT
    cp.Reg_no,
    cp.Course_code,
    cp.CA_Status,
    CASE WHEN att.eligibility IS NULL THEN 'Repeat student' ELSE
    att.eligibility END AS Attendance_Eligibility,
    CASE
    WHEN att.eligibility IS NULL THEN 'Eligible'
     WHEN 'MC' THEN 'MC'
        WHEN cp.CA_Status = 'CA_Pass' AND att.eligibility IN ('Eligible', 'Eligible with medical') THEN 'Eligible'
        WHEN cp.CA_Status = 'MC' AND att.eligibility IN ('Eligible', 'Eligible with medical') THEN 'Eligible'
        ELSE 'Not Eligible'
    END AS Final_Eligibility
FROM vw_CA_Status cp
LEFT JOIN vw_attend_percen_with_medical_with_elig_without_S_type_with_n att
    ON cp.Reg_no = att.reg_no AND cp.Course_code = att.course_code
ORDER BY cp.Reg_no, cp.Course_code;
```

```
DROP VIEW IF EXISTS vw_end_Status;
CREATE VIEW vw_end_Status AS
SELECT
    Reg_no,
    Course_code,
    end_marks,
    CASE
    WHEN END_marks = 'MC' THEN 'MC'
    WHEN END_marks = 'WH' THEN 'WH'
        WHEN end_marks = 'E*(Not_eligible)' THEN 'E*(Not_eligible)'
        WHEN Course_code IN ('ICT1222', 'ICT1233', 'ICT1253', 'TMS1233') THEN
            CASE WHEN end_marks >= 21 THEN 'END_Pass' ELSE 'END_Fail' END
        WHEN Course_code IN ('ENG1222', 'ICT1212', 'ICT1242', 'TCS1212') THEN
            CASE WHEN end_marks >= 25 THEN 'END_Pass' ELSE 'END_Fail' END
        ELSE 'Unknown'
    END AS END_Status
FROM (

    SELECT
        Reg_no,
        Course_code,
        `Total_end_marks(70%)` AS END_marks
    FROM vw_ENG1222_marks

    UNION ALL
```

```
DROP VIEW IF EXISTS vw_all_stu_all_course_grade_point;
CREATE VIEW vw_all_stu_all_course_grade_point AS
SELECT
    m.Reg_no,
    c.Course_Code AS Course,
    c.Credits AS Credit,
    m.`Total_CA_marks(30%)` AS CA_marks,
    m.`Total_End_marks(70%)` AS End_marks,
    m.`Final_Marks(100%)` AS Final_marks,
    ca_end.CA_Status,
    ca_end.END_Status,
    CASE
        WHEN ca_end.CA_Status = 'CA_Fail' AND ca_end.END_Status = 'END_Fail' THEN 'E(CA& ESA)'
        WHEN ca_end.CA_Status = 'CA_Fail' THEN 'E(CA)'
        WHEN ca_end.END_Status = 'END_Fail' THEN 'E(ESA)'
        WHEN m.`Final_Marks(100%)` = 'WH' THEN 'WH'
    WHEN m.`Final_Marks(100%)` = 'MC' THEN 'WH'
        WHEN s.Student_type = 'Repeat' AND m.`Final_Marks(100%)` >= 45 THEN 'C'
        WHEN s.Student_type = 'Repeat' AND m.`Final_Marks(100%)` >= 40 THEN 'C-'
        WHEN s.Student_type = 'Repeat' AND m.`Final_Marks(100%)` >= 35 THEN 'D'
        WHEN s.Student_type = 'Repeat' THEN 'E'
        WHEN m.`Final_Marks(100%)` >= 85 THEN 'A+'
        WHEN m.`Final_Marks(100%)` >= 75 THEN 'A'
        WHEN m.`Final_Marks(100%)` >= 70 THEN 'A-'
        WHEN m.`Final_Marks(100%)` >= 65 THEN 'B+'
        WHEN m.`Final_Marks(100%)` >= 60 THEN 'B'
        WHEN m.`Final_Marks(100%)` >= 55 THEN 'B-'
        WHEN m.`Final_Marks(100%)` >= 50 THEN 'C+'
        WHEN m.`Final_Marks(100%)` >= 45 THEN 'C'
        WHEN m.`Final_Marks(100%)` >= 40 THEN 'C-'
        WHEN m.`Final_Marks(100%)` >= 35 THEN 'D'
        ELSE 'E'
    END AS Grade,
    CASE

        WHEN m.`Final_Marks(100%)` = 'WH' THEN 'WH'
    WHEN m.`Final_Marks(100%)` = 'MC' THEN 'WH'
        WHEN s.Student_type = 'Repeat' AND m.`Final_Marks(100%)` >= 45 THEN 2.0
        WHEN s.Student_type = 'Repeat' AND m.`Final_Marks(100%)` >= 40 THEN 1.7
        WHEN s.Student_type = 'Repeat' AND m.`Final_Marks(100%)` >= 35 THEN 1.3
        WHEN s.Student_type = 'Repeat' THEN 0.0
        WHEN m.`Final_Marks(100%)` >= 85 THEN 4.0
        WHEN m.`Final_Marks(100%)` >= 75 THEN 4.0
        WHEN m.`Final_Marks(100%)` >= 70 THEN 3.7
```

```sql
DROP VIEW IF EXISTS vw_all_SGPA_and_CGPA;

CREATE VIEW vw_all_SGPA_and_CGPA AS
SELECT
    s.Reg_no,
    s.your_sgpa,
    c.`your cgpa`,
    c.class
FROM vw_all_stu_SGPA s
JOIN vw_all_stu_CGPA c ON c.Reg_no = s.Reg_no;
```

```sql
DROP VIEW IF EXISTS vw_all_stu_CGPA;
CREATE VIEW vw_all_stu_CGPA AS
    SELECT
        Reg_no,
        CASE
            WHEN has_mc > 0  THEN 'MC'
            WHEN has_wh > 0 THEN 'WH'
            WHEN course_count = 8 AND total_credits > 0 THEN
                ROUND(total_grade_points / total_credits, 2)
            ELSE 'No CGPA [repeat stu]'
        END AS `Your CGPA`,

        CASE
            WHEN course_count < 8 THEN 'Incomplete [repeat stu]'
            WHEN total_credits = 0 THEN 'No Valid Courses'
            WHEN has_mc > 0  THEN 'Incomplete(Medical Applied)'
            WHEN has_wh > 0 THEN 'WH'
            WHEN ROUND(total_grade_points / total_credits, 2) >= 3.70 THEN 'First Class'
            WHEN ROUND(total_grade_points / total_credits, 2) >= 3.30 THEN 'Second Class (Upper)'
            WHEN ROUND(total_grade_points / total_credits, 2) >= 3.00 THEN 'Second Class (Lower)'
            WHEN ROUND(total_grade_points / total_credits, 2) >= 2.00 THEN 'Pass'
            ELSE 'Fail'
        END AS `Class`
    FROM (
        SELECT
            Reg_no,
            COUNT(DISTINCT course) as course_count,
            SUM(CASE WHEN Grade = 'MC' THEN 1 ELSE 0 END) as has_mc,
        SUM(CASE WHEN Grade = 'WH' THEN 1 ELSE 0 END) as has_wh,
            SUM(CASE WHEN course != 'ENG1222' THEN credit * Grade_Point ELSE 0 END) as total_grade_points,
            SUM(CASE WHEN course != 'ENG1222' THEN credit ELSE 0 END) as total_credits
        FROM vw_all_stu_all_course_grade_point
        GROUP BY Reg_no
    ) as student_stats;
```

```sql
DROP VIEW IF EXISTS vw_all_stu_SGPA;
CREATE VIEW vw_all_stu_SGPA AS
SELECT
    Reg_no,
    CASE
        WHEN has_mc > 0 THEN 'MC'
        WHEN has_wh > 0 THEN 'WH'
        WHEN course_count = 8 AND total_credits > 0 THEN
            ROUND(total_grade_points / total_credits, 2)
        ELSE 'No SGPA [repeat stu]'
    END AS Your_SGPA
FROM (
    SELECT
        Reg_no,
        COUNT(DISTINCT course) as course_count,
        SUM(credit) as total_credits,
        SUM(credit * Grade_Point) as total_grade_points,
        SUM(CASE WHEN Grade = 'MC' THEN 1 ELSE 0 END) as has_mc,
        SUM(CASE WHEN Grade = 'WH' THEN 1 ELSE 0 END) as has_wh
    FROM vw_all_stu_all_course_grade_point
    GROUP BY Reg_no
) AS student_stats;
```

```
CREATE VIEW vw_result_sheet AS
SELECT
    a.Reg_no,
    (SELECT Grade FROM vw_all_stu_all_course_grade_point
        WHERE Reg_no = a.Reg_no AND course = 'ENG1222') AS ENG1222,
    (SELECT Grade FROM vw_all_stu_all_course_grade_point
        WHERE Reg_no = a.Reg_no AND course = 'ICT1212') AS ICT1212,
    (SELECT Grade FROM vw_all_stu_all_course_grade_point
        WHERE Reg_no = a.Reg_no AND course = 'ICT1222') AS ICT1222,
    (SELECT Grade FROM vw_all_stu_all_course_grade_point
        WHERE Reg_no = a.Reg_no AND course = 'ICT1233') AS ICT1233,
    (SELECT Grade FROM vw_all_stu_all_course_grade_point
        WHERE Reg_no = a.Reg_no AND course = 'ICT1242') AS ICT1242,
    (SELECT Grade FROM vw_all_stu_all_course_grade_point
        WHERE Reg_no = a.Reg_no AND course= 'ICT1253') AS ICT1253,
    (SELECT Grade FROM vw_all_stu_all_course_grade_point
        WHERE Reg_no = a.Reg_no AND course = 'TCS1212') AS TCS1212,
    (SELECT Grade FROM vw_all_stu_all_course_grade_point
        WHERE Reg_no = a.Reg_no AND course = 'TMS1233') AS TMS1233,
    b.your_sgpa,
    b.`your cgpa`,
    b.class
FROM vw_all_sgpa_and_cgpa b
JOIN vw_all_stu_all_course_grade_point a
    ON a.Reg_no = b.Reg_no
GROUP BY a.Reg_no;
```

❖ list of All Procedures

```
mysql> SELECT ROUTINE_NAME
    -> FROM information_schema.ROUTINES
    -> WHERE ROUTINE_TYPE = 'PROCEDURE'
    -> AND ROUTINE_SCHEMA = 'faculty_of_technology';
+-----------------------------------------------------+
| ROUTINE_NAME                                        |
+-----------------------------------------------------+
| CA_marks_all_sub_by_regNo                           |
| GetAttendanceByCourseCodeWithMedical                |
| GetAttendanceByCourseCodeWithoutMedical             |
| GetAttendanceByCourseCodeWithSTypeWithMedical       |
| GetAttendanceByCourseCodeWithSTypeWithoutMedical    |
| GetAttendanceByRegnoWithMedical                     |
| GetAttendanceByRegnoWithoutMedical                  |
| GetAttendanceByRegnoWithSTypeWithMedical            |
| GetAttendanceByRegnoWithSTypeWithoutMedical         |
| GetFinalMarksByCourseCode                           |
| GetStudentFinalMarks                                |
| get_attend_by_regno_and_coursecode_without_stype    |
| get_attend_by_regno_and_coursecode_with_stype       |
| give_course_get_CA                                  |
| give_course_reg_get_CA                              |
| individual_SGPA_and_CGPA                            |
+-----------------------------------------------------+
16 rows in set (0.00 sec)
```

- procedures

```
DELIMITER //

CREATE PROCEDURE GetAttendanceByCourseCodeWithoutMedical(IN course_code_param VARCHAR(10))
BEGIN
    SELECT
    course_code,
        reg_no,
        SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) AS attended_hours,
        SUM(session_hours) AS total_hours,
        SUM(CASE WHEN status = 'Present' THEN 1 ELSE 0 END) AS attended_sessions,
    COUNT(*) AS total_sessions,
        ROUND(
            (SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) /
             SUM(session_hours)) * 100, 2
        ) AS attendance_percentage,
    CASE
            WHEN (SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) / SUM(session_hours)) >= 0.8
                THEN 'Eligible'
                ELSE 'Not Eligible'
        END AS eligibility
    FROM attendance
    WHERE course_code = course_code_param
```

```
DROP PROCEDURE IF EXISTS GetAttendanceByCourseCodeWithSTypeWithoutMedical;

DELIMITER //

CREATE PROCEDURE GetAttendanceByCourseCodeWithSTypeWithoutMedical(IN course_code_param VARCHAR(10))
BEGIN
    SELECT
    course_code,
        reg_no,
    session_type,
        SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) AS attended_hours,
        SUM(session_hours) AS total_hours,
        SUM(CASE WHEN status = 'Present' THEN 1 ELSE 0 END) AS attended_sessions,
    COUNT(*) AS total_sessions,
        ROUND(
            (SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) /
            SUM(session_hours)) * 100, 2
        ) AS attendance_percentage,
```

```
DROP PROCEDURE IF EXISTS GetAttendanceByCourseCodeWithMedical;

DELIMITER //

CREATE PROCEDURE GetAttendanceByCourseCodeWithMedical(IN course_code_param VARCHAR(10))
BEGIN
SELECT
    a.course_code,
    a.reg_no,
    SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END) AS attended_hours,
    SUM(session_hours) AS total_hours,
    SUM(CASE WHEN a.status = 'Present' THEN 1  WHEN a.status='Medical' AND m.approved=TRUE THEN 1 ELSE 0 END) AS attended_sessions,
    COUNT(*) AS total_sessions,
    ROUND( (
 SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.session_hours)
    ) * 100, 2
    ) AS attendance_percentage,
    CASE WHEN
 SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.ses
```

```
DROP PROCEDURE IF EXISTS GetAttendanceByCourseCodeWithSTypeWithMedical;

DELIMITER //

CREATE PROCEDURE GetAttendanceByCourseCodeWithSTypeWithMedical(IN course_code_param VARCHAR(10))
BEGIN
SELECT
    a.reg_no,
    a.course_code,
    a.session_type,
    SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END) AS attended_hours,
    SUM(session_hours) AS total_hours,
    SUM(CASE WHEN a.status = 'Present' THEN 1  WHEN a.status='Medical' AND m.approved=TRUE THEN 1 ELSE 0 END) AS attended_sessions,
    COUNT(*) AS total_sessions,
    ROUND( (
 SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.session_hours)
    ) * 100, 2
    ) AS attendance_percentage,
    CASE WHEN
```

```
DELIMITER //

CREATE PROCEDURE GetAttendanceByRegnoWithSTypeWithMedical(IN input_reg_no VARCHAR(20))
BEGIN
SELECT
    a.reg_no,
    a.course_code,
    a.session_type,
    SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END) AS attended_hours,
    SUM(session_hours) AS total_hours,
    SUM(CASE WHEN a.status = 'Present' THEN 1  WHEN a.status='Medical' AND m.approved=TRUE THEN 1 ELSE 0 END) AS attended_sessions,
    COUNT(*) AS total_sessions,
    ROUND( (
SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.session_hours)
    ) * 100, 2
    ) AS attendance_percentage,
    CASE WHEN
SUM(CASE WHEN a.status = 'Present' THEN a.session_hours  WHEN a.status='Medical' AND m.approved=TRUE THEN a.session_hours ELSE 0 END)/ SUM(a.session_hours)
    >= 0.8 AND SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) / SUM(session_hours) < 0.8 THEN 'Eligible with medical'
WHEN (SUM(CASE WHEN status = 'Present' THEN session_hours ELSE 0 END) / SUM(session_hours)) >= 0.8
            THEN 'Eligible'
       ELSE 'Not Eligible'
```

```
DELIMITER //

CREATE PROCEDURE CA_marks_all_sub_by_regNo (IN TG CHAR(20))
BEGIN
    SELECT Reg_no, 'ENG1222' AS Course,`Total_CA_marks(30%)` AS CA_marks FROM vw_ENG1222_marks WHERE Reg_no = TG
    UNION ALL
    SELECT Reg_no, 'ICT1212' AS Course,`Total_CA_marks(30%)` AS CA_marks FROM vw_ICT1212_marks WHERE Reg_no = TG
    UNION ALL
    SELECT Reg_no, 'ICT1222' AS Course,`Total_CA_Marks(40%)` AS CA_marks FROM vw_ICT1222_marks WHERE Reg_no = TG
    UNION ALL
    SELECT Reg_no, 'ICT1233' AS Course,`Total_CA_Marks(40%)` AS CA_marks FROM vw_ICT1233_marks WHERE Reg_no = TG
    UNION ALL
    SELECT Reg_no, 'ICT1242' AS Course,`Total_CA_marks(30%)` AS CA_marks FROM vw_ICT1242_marks WHERE Reg_no = TG
    UNION ALL
    SELECT Reg_no, 'ICT1253' AS Course,`Total_CA_marks(40%)` AS CA_marks FROM vw_ICT1253_marks WHERE Reg_no = TG
    UNION ALL
    SELECT Reg_no, 'TCS1212' AS Course,`Total_CA_marks(30%)` AS CA_marks FROM vw_TCS1212_marks WHERE Reg_no = TG
    UNION ALL
    SELECT Reg_no, 'TMS1233' AS Course,`Total_CA_marks(40%)` AS CA_marks FROM vw_TMS1233_marks WHERE Reg_no = TG;
END //

DELIMITER ;
```

```
DROP PROCEDURE IF EXISTS get_attend_by_regno_and_coursecode_with_stype ;

DELIMITER //
CREATE PROCEDURE get_attend_by_regno_and_coursecode_with_stype(IN TG CHAR(20),IN cid CHAR(10))

BEGIN
SELECT * FROM vw_attend_percentage_with_medical_with_eligibility_with_name  WHERE Reg_no = TG AND
course_code = cid;
END //

DELIMITER ;
```

```
DROP PROCEDURE IF EXISTS get_attend_by_regno_and_coursecode_without_stype;

DELIMITER //

CREATE PROCEDURE get_attend_by_regno_and_coursecode_without_stype(IN TG CHAR(20),IN cid CHAR(10))
BEGIN
SELECT * FROM vw_attend_percen_with_medical_with_elig_without_S_type_with_n  WHERE Reg_no = TG AND
course_code = cid;
END //

DELIMITER ;
```

```
DROP PROCEDURE IF EXISTS give_course_get_CA;
DELIMITER //

CREATE PROCEDURE give_course_get_CA (IN p_course_code CHAR(20))
BEGIN
    IF p_course_code = 'TMS1233' THEN
        SELECT
            m.Reg_no,
            m.Course_code,
            m.`Quiz_1(100%)`,
            m.`Quiz_2(100%)`,
            m.`Mid_theory(100%)`,
            m.`Total_CA_Marks(40%)`,
            c.CA_Status
        FROM vw_TMS1233_marks m
        JOIN vw_CA_Status c ON m.Reg_no = c.Reg_no AND m.Course_code = c.Course_code;
    ELSEIF p_course_code = 'TCS1212' THEN
        SELECT
            m.Reg_no,
            m.Course_code,
            m.`Quiz_1(100%)`,
            m.`Assignment_1(100%)`,
            m.`Mid_theory(100%)`,
            m.`Total_CA_Marks(30%)`,
            c.CA_Status
        FROM  vw_TCS1212_marks m
        JOIN vw_CA_Status c ON m.Reg_no = c.Reg_no AND m.Course_code = c.Course_code;
     ELSEIF p_course_code = 'ICT1253' THEN
        SELECT
            m.Reg_no,
            m.Course_code,
            m.`Max_Quiz(100%)`,
            m.`Second_Max_Quiz(100%)`,
            m.`Assignment_1(100%)`,
```

```
DELIMITER //

CREATE PROCEDURE GetFinalMarksByCourseCode(IN course_code VARCHAR(20))
BEGIN
    SELECT
        g.Course AS CourseCode,
        g.Reg_no,
        g.Final_marks AS FinalMark,
        g.Grade,
        CASE
            WHEN g.Grade IN ('A+', 'A', 'A-', 'B+', 'B', 'B-', 'C+', 'C', 'C-', 'D') THEN 'Pass'
            WHEN g.Grade = 'WH' THEN 'Withheld'
            WHEN g.Grade = 'MC' THEN 'MC'
            WHEN g.Grade LIKE 'E(%' THEN 'Fail'
            WHEN g.Grade = '_' THEN 'Not Eligible for End Exam'
            ELSE 'Fail'
        END AS FinalStatus
    FROM vw_all_stu_all_course_grade_point g
    WHERE g.Course = course_code
    ORDER BY g.Reg_no;
END //

DELIMITER ;
```

```
DELIMITER //

CREATE PROCEDURE GetStudentFinalMarks(IN reg_no CHAR(20))
BEGIN
    SELECT
        g.Reg_no,
        g.Course AS CourseCode,
        g.Final_marks AS FinalMark,
        g.Grade,
        CASE
            WHEN g.Grade IN ('A+', 'A', 'A-', 'B+', 'B', 'B-', 'C+', 'C', 'C-', 'D') THEN 'Pass'
            WHEN g.Grade = 'WH' THEN 'Withheld'
            WHEN g.Grade = 'MC' THEN 'MC'
            WHEN g.Grade LIKE 'E%' THEN 'Fail'
            WHEN g.Grade = '_' THEN 'Not Eligible'
            ELSE 'Pending'
        END AS FinalStatus
    FROM vw_all_stu_all_course_grade_point g
    WHERE g.Reg_no = reg_no
    ORDER BY g.Course;
END //

DELIMITER ;
```

❖ Queries

```
-- medical id correction of attendance table
UPDATE attendance SET Medical_id = NULL;
UPDATE attendance SET Status = 'Present', Medical_id = NULL;
UPDATE attendance a
INNER JOIN medical_record m
ON a.Reg_no = m.reg_no AND a.Session_date = m.session_date
SET a.Status = 'Medical',
    a.Medical_id = m.Medical_id;

-- correcting aa status for better
UPDATE attendance
SET Status = 'Absent'
WHERE Status = 'Present'
AND Medical_id IS NULL
AND (Attendance_id % 7 = 0)
LIMIT 180;
```

XI. Problems that were faced during the development of the solution

- Create sample data for every student. It is challenging to create data for 15 weeks because some tables contain thousands of rows.
- Find the correct marks criteria of each course.
- Various types of exams per course.
- Some logics are very complex and difficult to create.
- efficiently collaborate with group members.

XII. Solutions/how have we overcome the above-identified problems

- Group members create data by sharing. All group members create that data by sharing.
- Examine UGC Commission Circular No. 12-2024 for getting updated with the criteria.
- Normalized database
- Use SQL constraints for data validations.

XIII. Choose a platform for the host database and the reason

I will choose a cloud provider like,

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud Platform (GCP) for the following

Because then,

- Easily increase or decrease server resources
- High Availability
- Provides built-in security features like encryption and access control
- Provide backups
- Can pay only for the resources we use

So, it is a better choice for hosting.

XIV. The things/changes to do for the host database in cloud storage

- Move our MySQL database from the local computer to a cloud database service.
- Change all connection strings in our application to connect to the cloud database
- Set up cloud firewall rules and security groups to control who can access the database.

XV. Individual contributions

Github Repository Link - git@github.com:dilusharadeeshan/Student-Management-System.git

TG/2023/1721
P.H.T. Sharindi

For my contribution to the database design and development, I created key tables, views, and stored procedures.

Tables & Data Insertion part

1. **ATTENDANCE –** Store each student's presence details for every course session. The Attendance_id is the Primary Key (PK), which uniquely identifies each attendance record. It includes fields like Reg_no, Course_code, Session_type, Session_hours, Status. The Reg_no, Course_code, and Medical_id act as Foreign Kry (FK) that link this table with student, course, and medical_record tables.(Insert data to attendance table(from 1 week to 5 week))

2. **MEDICAL_RECORD –** Stores the medical information of students who are absent due to health reasons. Primary Key is Medical_id, and it connects back to the attendance table through a foreign key relationship.(Insert data to medical_record table from 08/05 to 08/30 of 2025)

3. **ENROLL_IN -** insert data to enroll in table(006-010)

Views

1. **vw_eligibi_without_medical_with_name –** This SQL view calculates student attendance eligibility for each course and session type, without **considering medical leave.**
2. **vw_eligibi_without_medical_with_name_without_s_type –** This SQL view calculates student attendance eligibility for each course (ignoring session types), without considering medical leave.
3. **vw_end_exam_elig_basedon_ca_and_atten** – This SQL view determines final eligibility for end-of-semester exam by combining both CA status and attendance eligibility.
4. **vw_ict1233_marks –** This view acts like a grade calculator specifically for the ICT1233 course, showing exactly how each student's final grade is determined.

5. **vw_ict1242_marks** – This SQL view calculates detailed marks for the ICT1242 course,

breaking down quizzes, mid-terms, assignments, and finals

6. **vw_total_ca_end_status –** This SQL view combines CA and End exam eligibility statuses to determine if students' quality for finals.
7. **vw_all_sgpa_and_cgpa –** This SQL summarizes overall academic performance, showing semester and GPAs with class rankings for each student.
8. **vw_result_sheet –** This SQL displays a complete academic transcript with grades for all courses SGPA and CGPA.

Procedures

1. **GetAttendanceByCourseCodeWithSTypeWithMedical**

- This stored procedure retrieves attendance records for a specific course while considering approved medical leaves. It calculates attendance percentage and eligibility for end exams, where medical absences are counted as present if approved.
- CALL GetAttendanceByCourseCodeWithSTypeWithMedical('ICT1242');

2. **GetAttendanceByCourseCodeWithSTypeWithoutMedical**

- Shows attendance for a course counting only actual "Present" records. Medical leaves are ignored. Determines if students meet the strict 80% requirement for exam eligibility.
- CALL GetAttendanceByCourseCodeWithSTypeWithoutMedical('ICT1233');

3. **get_attend_by_regno_and_coursecode_with_stype**

- Shows one student's detailed attendance for a specific course, broken down by session types (Lecture/Lab). Includes approved medical leaves in the attendance count.
- CALL get_attend_by_regno_and_coursecode_with_stype('TG/2023/003','ICT1233');

4. **CA_marks_all_sub_by_regNo**

- Shows all CA marks for one student across every subject in a single report.
- CALL CA_marks_all_sub_by_regNo('ICT1242');

5. **GetFinalMarksByCourseCode**

- Shows final exam results for all students in a specific course, including their marks, grades, and pass/fail status.
- CALL GetFinalMarksByCourseCode('ICT1242');

Users

Admin - With All privileges with Grant Option for all the tables in the database

Dean - With All privileges without Grant for all the tables in the database

Lecturer - All privileges without Grant and user creation for all the tables in the database

TG/2023/1748
R.P.D.Radeeshan

- Created tables
    - **User table** and subclass tables of it. They are the **admin table**, **dean table**, **student table**, **technical_officer table**, and **lecturer table**.
    - **Enrollment table** for the M-N relationship between the course and the student.

- Created data
    - **Created data for the user table,** including one dean, one admin, 7 lecturers, 7 technical officers, 15 proper students, and 8 repeat students for the insert user table and subclass of it.
    - **Created some data for the enrollment** table to cover 5 proper students and all repeat students.
    - **Created some data for attendance table** for all students to cover 2025/09/08 to 2025/10/10.
    - **Created some data for the mark table** to cover 5 proper students and all repeat student based on enrollment for the course.
    - **Created some data for medical table** to cover medical that apply 2025/08/04 to 2025/08/22.

- Created logic
    - Created logic/wrote a **query for updating attendance table data for matching medical** record table data. By it correct column medical id of the attendance table matches the medical record based on date and regno.
    - Created **query to update the attendance table data** for attendance percentage matching around 80% percentages.
    - Created **query for correct data of mark table** to match eligibility for end exam.

- Created views
    - Create a view **vw_medicalApplyAttendance** to see medical applied attendance.
    - Create a view **vw_approvedMedical** to see approved medical .
    - Created a view **vw_attend_percen_with_medical_with_elig_without_s_type_with_n** to look at the attendance percentage with medical approval and eligibility for the end exam

without considering the session type, and Created a view **vw_attend_percentage_with_medical_with_eligibility_with_name** with considering the session type .

- Created a view **vw_eng1222_marks** for looking at CA marks details, End marks details, and also final marks details for the subject 'ENG1222' .
- Created a view **vw_ict1212_marks** for looking at CA marks details, End marks details, and also final marks details for the subject 'ICT1212' .
- Created a view **vw_ict1222_marks** for looking at CA marks details, End marks details, and also final marks details for the subject 'ICT1222' .
- Created a view **vw_ca_status** to look at CA marks and CA pass fail status for the whole badge for every course .
- Created a view **vw_all_stu_all_course_grade_point** for CA marks, End marks, Final marks, CA status, End status with grade and grade point for the whole batch for all courses.

- Procedures
  - Created procedure **GetAttendanceByCourseCodeWithMedical** to get attendance with medical aproval with eligibility to whole batch for particular course by giving course code.
  - Created procedure **GetAttendanceByRegnoWithSTypeWithMedical** to get attendance with medical aproval with eligibility with session type to whole courses by giving reg no .
  - Created procedure **give_course_get_CA** to get all CA marks details with CA_status by giving course code for whole batch .
  - Created procedure **GetStudentFinalMarks** to get final marks details with final_status and grade by giving reg no for whole courses .
  - Created procedure **get_attend_by_regno_and_coursecode_without_stype** to get attendance details with eligibility by giving course code and reg no .

- Users
  - **Create a technical officer user** with Read, write, and update permissions for attendance-related tables/views like ,

    attendance, vw_end_exam_elig_basedon_ca_and_atten, vw_eligibi_without_medical_with_name_without_s_type, vw_attend_percentage_with_medical_with_eligibility_with_name, etc.

TG/2023/1781
H.G.D.I Shadeep

The first part of our project is to draw the er and relational mapping .I also provided my support that. For our Student Management System DBMS Project , I Controbuted by developing portion of the database table , such as mark , course etc… In Addition to this I designed and develop several SQL views and stored procedures.

Table & Data insertion

1. Course - Store information about university courses
2. Mark - The System stores Data releted to the student's Reg_no , course_code and all the exam types they face.
3. Enter the Data releted to the course table.
4. A large amount of data was entered for the enrollment table and some of it has been inserted.
5. furthermore A large amount of data was entered for the Attendance table and some of it has been inserted.
6. A large amount of data was entered for the Medical_record table and some of it has been inserted.
7. Additionaly I was enterd portion of data to mark table .

Views

1. **vw_attend_with_medical_approval**

provide the session details relevant to the student , that combines data from the attendance and medical_record tables .from this ,we can see whether a students's medical is approved or not approved .

2. **vw_tcs1212_marks , vw_tms1212_marks ,vw_ict1253_marks**

This shows the quiz marks , assignment marks , mid marks , end marks , total CA marks , final exam marks , and the overall marks related to each course codes .

3. **vw_all_stu_sgpa , vw_all_stu_cgpa**

Through these two views , we can obtain in CGPA , CLASS and SGPA for all registration numbers .

Procedures

1.GetAttendanceByRegnoWithMedical , GetAttendanceByRegnoWithoutMedical

These procedures display all the courses related to a specific registration number , showing the attended hours , total hours , attended sessions , attended percentage and eligibility both with and without mecical consideration.

2.GetAttendanceByCourseCodeWithoutMedical

This show all the above details for all students related to a specific course.

3.GetAttendanceByRegnoWithSTypeWithoutMedical

For a given registration number ,show for all enrolled courses the session type , attended hours , total hours , total sessions , attended sessions ,attendance percentage and eligibility .

4.give_course_reg_get_CA

For a student ,after entering their registration number and the relevant course , display the maximum quiz , second maxmum quiz ,assignment mark ,mid mark ,total CA mark and CA status .

5.individual_SGPA_and_CGPA

For a student,after entering their registration number, they can view their CGPA,SGPA, and CLASS.

Users

I created a user (student) and gave permission to only select data from the mark and attendance tables . I also gave the student permission to access the view and procedures related to marks and attendance

XVI. References

- W3Schools. MySQL Tutorial. Available at: https://www.w3schools.com/mysql/
- MySQL Documentation. Official Reference Manual. Available at: https://dev.mysql.com/doc/
- freeCodeCamp. MySQL Tutorials and Guides. Available at: https://www.freecodecamp.org/
- Lecture Notes. Course materials provided by the instructor.